

Arquitectura

Decisiones Técnicas y Trade-offs

DistributedProcessing, Documento 02-arquitectura

1. Resumen ejecutivo

Este documento describe las decisiones arquitectónicas principales del proyecto DistributedProcessing y los trade-offs evaluados en cada elección. El proyecto implementa un sistema de procesamiento distribuido utilizando Go, gRPC y Docker.

2. Por qué Go

2.1. Decisión

Go (Golang) fue seleccionado como el lenguaje principal para implementar el sistema de procesamiento distribuido.

2.2. Justificación

Go proporciona **goroutines**, threads ligeros manejados por el runtime, capaces de ejecutar miles o millones de instancias simultáneamente sin el overhead de los threads del sistema operativo, lo que lo hace ideal para sistemas distribuidos que manejan múltiples conexiones concurrentes.

El lenguaje compila a un único binario estático sin dependencias externas, lo que simplifica la distribución a múltiples nodos y encaja de forma natural con la contenedorización en Docker, reduciendo el tamaño de las imágenes generadas.

En cuanto a rendimiento, Go es casi tan rápido como C o C++ pero considerablemente más fácil de mantener, con bajo consumo de memoria y un recolector de basura optimizado para baja latencia, lo cual resulta apropiado para sistemas con recursos limitados.

Finalmente, Go cuenta con un ecosistema robusto para sistemas distribuidos: soporte nativo y de primera clase para gRPC, librerías de red maduras como net, net/http y context, y herramientas de testing integradas para código concurrente.

2.3. Trade-offs

Aspecto	Ventaja	Desventaja
Curva de aprendizaje	Sintaxis simple, fácil de aprender	Si el equipo viene de OOP, requiere cambio de paradigma
Ecosistema	Grande para sistemas distribuidos	Menos librerías que Python o Java para ML/IA

Aspecto	Ventaja	Desventaja
Tipado	Tipado estático, mejor seguridad	Menos flexible que lenguajes dinámicos

2.4. Alternativas evaluadas

Se evaluó Python, descartado por ser más lento y por el overhead del GIL para concurrencia real. Se evaluó Java, descartado por el peso de la JVM y su mayor consumo de memoria. Se evaluó Rust, que ofrece mayor seguridad pero con una curva de aprendizaje más pronunciada que Go.

3. Por qué gRPC

3.1. Decisión

gRPC fue seleccionado como el protocolo RPC para la comunicación entre servicios distribuidos.

3.2. Justificación

gRPC utiliza Protocol Buffers para una serialización binaria compacta y rápida, y se apoya en HTTP/2 con multiplexing nativo, permitiendo múltiples streams en una sola conexión y reduciendo significativamente el ancho de banda frente a REST sobre JSON.

En cuanto a rendimiento, gRPC ofrece latencia baja y throughput alto para comunicación de alta frecuencia entre servicios, siendo típicamente uno o dos órdenes de magnitud más rápido que REST para carga de datos.

Los archivos .proto definen contratos de interfaz explícitos con tipado fuerte, lo que permite validación automática de mensajes y elimina la serialización manual mediante generación de código. Además, gRPC soporta streaming bidireccional de forma nativa, habilitando patrones de mensajería más complejos entre cliente y servidor.

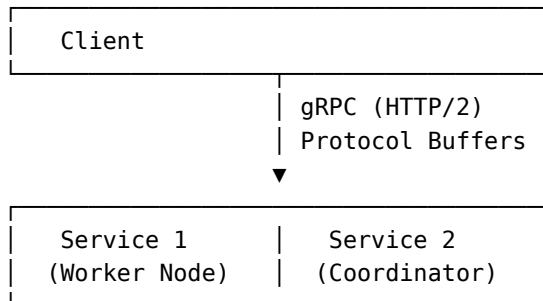
3.3. Trade-offs

Aspecto	Ventaja	Desventaja
Curva de aprendizaje	Potente y eficiente	Requiere aprender Protocol Buffers
Debugging	Tipado fuerte, errores claros	Mensajes binarios, no legibles en texto plano
Browser	No requiere HTTP/1.1	No soporta llamadas directas desde el navegador sin gRPC-Web
Ecosistema REST	Herramientas especializadas	Menos omnipresente que REST en todos los lenguajes

3.4. Alternativas evaluadas

Se evaluó REST sobre JSON, más simple pero menos eficiente para comunicación de alta frecuencia. Se evaluó GraphQL, más flexible pero con overhead de parsing. Se evaluaron colas de mensajes como RabbitMQ o Kafka, adecuadas para asincronía pero más complejas para RPC sincrónico. Se evaluaron WebSockets, viables pero menos optimizados que HTTP/2 combinado con gRPC.

3.5. Arquitectura de servicios con gRPC



4. Por qué Docker

4.1. Decisión

Docker fue seleccionado para la contenedorización y orquestación de componentes del sistema.

4.2. Justificación

Docker garantiza portabilidad: el mismo contenedor corre de forma idéntica en desarrollo, testing y producción, eliminando el problema de «funciona en mi máquina» al encapsular completamente las dependencias, de forma multiplataforma entre Linux, Windows y Mac.

Cada contenedor corre en su propio namespace de procesos, red y sistema de archivos, ofreciendo aislamiento de recursos, un entorno reproducible y totalmente versionable, y la posibilidad de correr múltiples versiones del mismo servicio de forma independiente.

Docker facilita además la orquestación posterior: es el estándar de facto junto con Kubernetes, permite agregar o remover instancias de forma trivial, y se integra naturalmente con sistemas de despliegue distribuido.

La combinación de Go y Docker resulta particularmente favorable, ya que Go compila binarios pequeños, típicamente entre 5 y 50 MB, lo que permite imágenes muy ligeras partiendo de scratch y sin necesidad de un runtime adicional como la JVM o un intérprete de Python.

4.3. Ejemplo: Dockerfile óptima para Go

```
# Etapa 1: Build
FROM golang:1.26.1 AS builder
WORKDIR /app
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o main .
```

```
# Etapa 2: Runtime (Multi-stage)
```

```
FROM scratch
COPY --from=builder /app/main /main
EXPOSE 50051
CMD ["/main"]
```

El resultado es una imagen de aproximadamente 10 MB, sin dependencias adicionales.

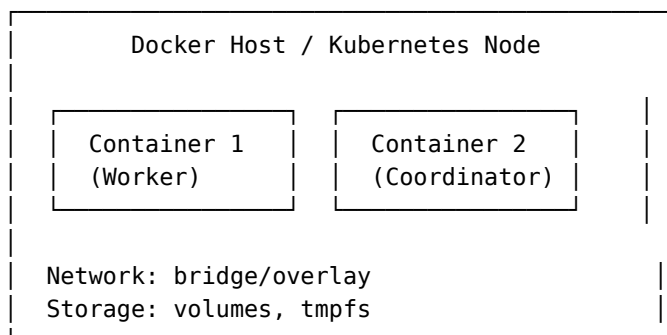
4.4. Trade-offs

Aspecto	Ventaja	Desventaja
Overhead	Mínimo en Linux nativo	Overhead en Mac/Windows con Docker Desktop
Curva de aprendizaje	Conceptos simples	Requiere entender contenedores y networking
Debugging	Aislamiento seguro	Más complejo depurar dentro de un contenedor
Persistencia	Volúmenes bien soportados	Requiere gestión explícita de estado

4.5. Alternativas evaluadas

Se evaluaron máquinas virtuales tradicionales, con más aislamiento pero un overhead masivo en gigabytes frente a megabytes. Se evaluó despliegue en bare metal, con máximo rendimiento pero complejidad de despliegue extrema. Se evaluaron plataformas serverless, descartadas por la falta de control sobre el runtime y la latencia fría.

4.6. Arquitectura de despliegue



5. Matriz de decisiones: evaluación de alternativas

Criterio	Go	Java	Python	Rust
Concurrencia	5/5	4/5	2/5	5/5
Performance	5/5	4/5	3/5	5/5

Criterio	Go	Java	Python	Rust
Tamaño binario	5/5	2/5	2/5	5/5
Curva de aprendizaje	5/5	3/5	5/5	3/5
Ecosistema distribuido	5/5	4/5	3/5	3/5

6. Arquitectura general del sistema

El siguiente diagrama, en notación Mermaid, resume la arquitectura general: los clientes acceden a un balanceador de carga sobre la capa gRPC, que distribuye las peticiones entre los Workers, coordinados por el Master, y cada componente corre en su propio contenedor Docker.

```
graph TB
    subgraph clients["Clients"]
        C1[Client 1]
        C2[Client N]
    end

    subgraph grpc["gRPC Layer (HTTP/2 + Protocol Buffers)"]
        LB[Load Balancer]
    end

    subgraph services["Compute Nodes"]
        S1["Worker Service 1<br/>(Go + gRPC)"]
        S2["Worker Service N<br/>(Go + gRPC)"]
        COORD["Coordinator<br/>(Go + gRPC)"]
    end

    subgraph docker["Docker / Kubernetes"]
        D1["Container 1"]
        D2["Container N"]
        DCOORD["Container Coordinator"]
    end

    C1 --> LB
    C2 --> LB
    LB --> S1
    LB --> S2
    S1 --> COORD
    S2 --> COORD
    S1 -. -> D1
    S2 -. -> D2
    COORD -. -> DCOORD
```

7. Conclusiones y próximos pasos

Las decisiones de usar Go, gRPC y Docker quedan ratificadas: Go resulta ideal para sistemas distribuidos con alta concurrencia, gRPC ofrece máxima eficiencia en la comunicación entre servicios, y Docker garantiza portabilidad y escalabilidad.

Como consideraciones futuras se contempla implementar Kubernetes para orquestación automática, incorporar observabilidad mediante distributed tracing con Jaeger y métricas con Prometheus, implementar patrones de circuit breaker y retry logic en gRPC, y configurar health checks en los servicios.

8. Referencias

Documentación oficial de Go en golang.org/doc, guía de gRPC en Go en grpc.io/docs/languages/go, especificación de Protocol Buffers en developers.google.com/protocol-buffers, buenas prácticas de Docker en docs.docker.com/develop/dev-best-practices y documentación de Kubernetes en kubernetes.io/docs.